

Digital Design with Verilog

FSM Design Examples with Verilog

A. Sahu

Dept of Comp. Sc. & Engg.

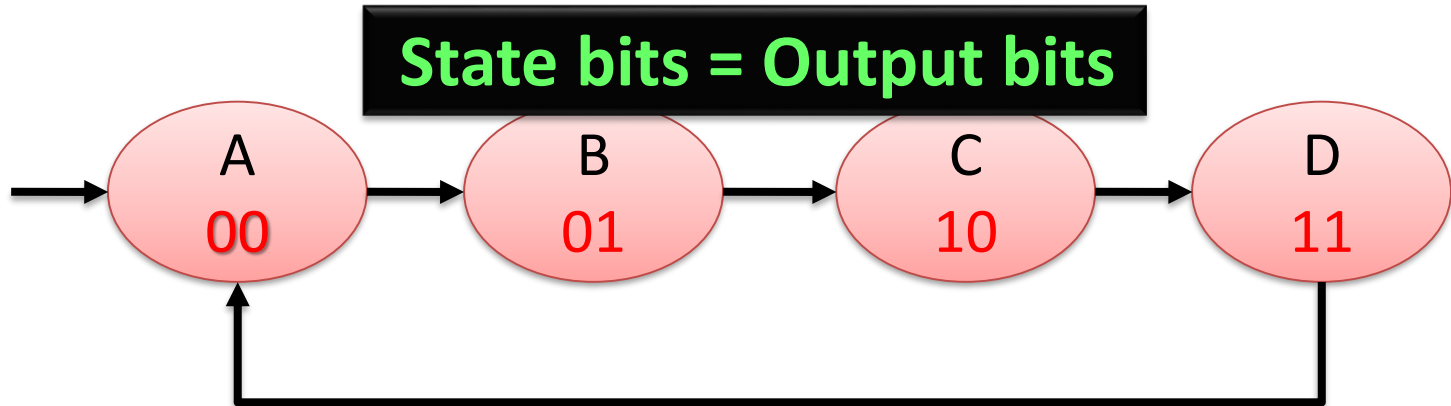
Indian Institute of Technology Guwahati

FSM Example 5

(General : Two bit binary Counter)

Two bit counter: 00,01,10,11,repeat

- Step1: Want 00, 01, 10, 11, repeat
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states, Transition on rising clock edge to next state



FSM of Counter : 2 bit

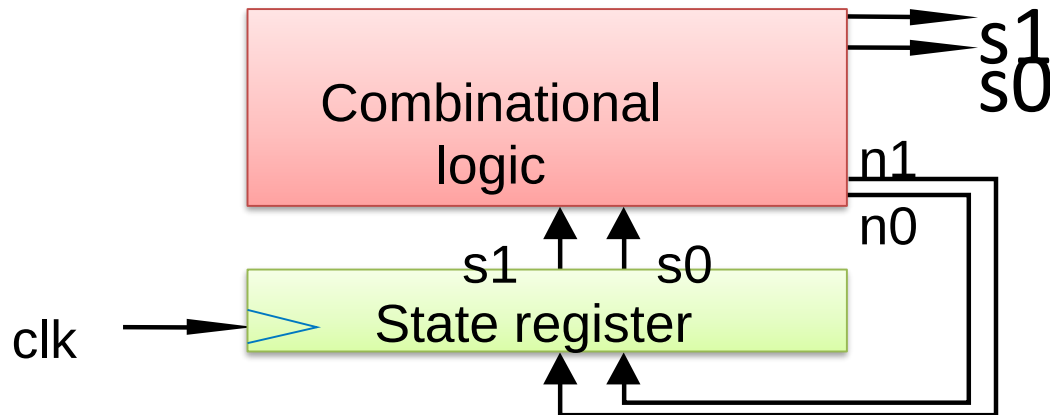
Step 3: Represent the FSM in Tabular Form

Step 4: Derive number of FF require

Four States: 2 FF : Obvious

Step 5: State Encoding

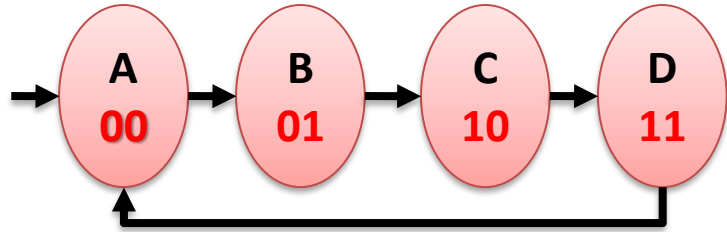
State A: 00 State B: 01,
State C: 10, State D: 11



	I/P		O/P	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
B	1	1	0	0

Step 6: Derivation of Next State

FSM Controller: Binary Counter



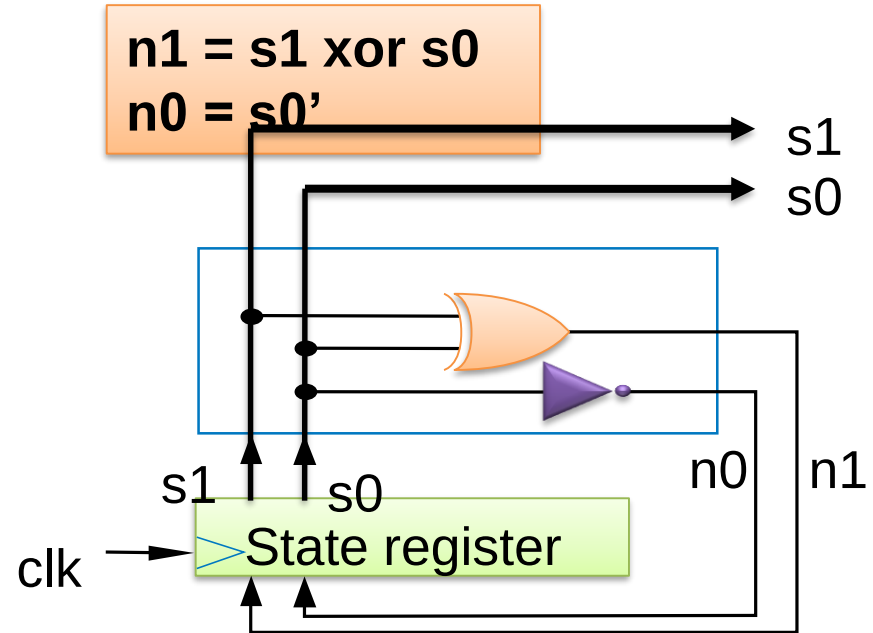
Step 7: Derive for NS logic, OP logic

NS/OP : $n1 = s1 \text{ xor } s0$; $n0 = s0'$

	I/P		O/P	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
D	1	1	0	0

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock

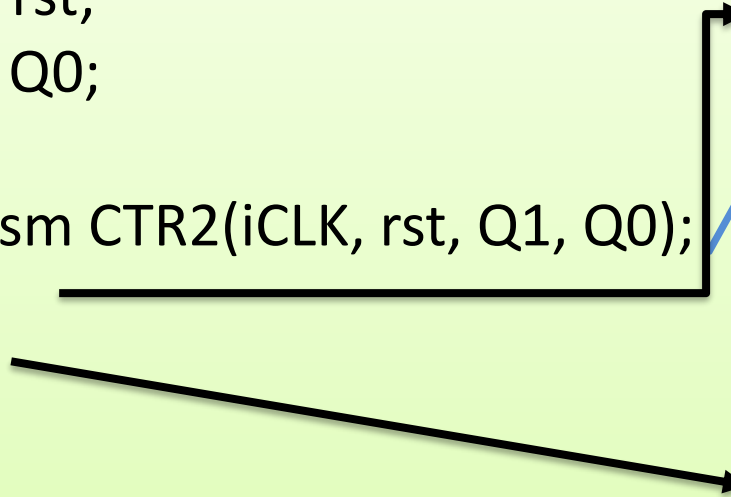


Verilog Code for FSM of 2 bit CTR

```
module ctr2bit_fsm (input clk, input rst, output q1, output q0);  
    wire NS0, NS1, NS2, S0, S1, S2;  
  
    DFF D0(NS0, clk, rst, S0); //Instantiation  
    DFF D1(NS1, clk, rst, S1); //Instantiation  
  
    assign NS1 = S1 ^ S0; //NS logic  
    assign NS0 = ~S0;      //NS logic  
    assign q1 = S1;        //output logic  
    assign q0 = S0;        //output logic  
  
endmodule
```

Verilog Code for FSM of Ctr: Test bench

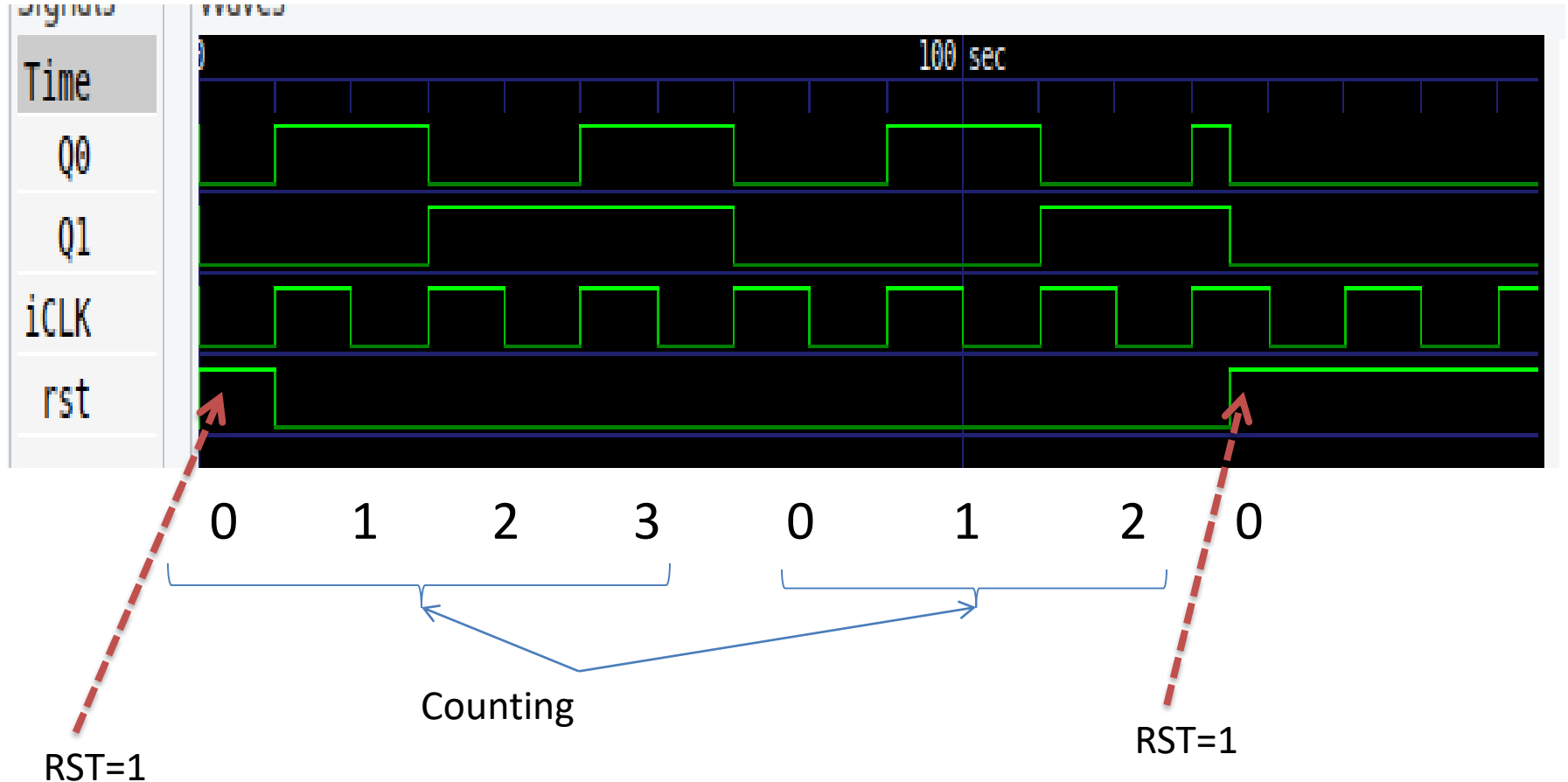
```
module 2bitctr_fsm_test;
reg iCLK, rst;
wire Q1, Q0;
integer i;
ctr2bit_fsm CTR2(iCLK, rst, Q1, Q0); //
//Part I
//Part II
initial
    $monitor ("iCLK = %b, Q1 = %b, Q0 = %b",
              iCLK, Q1, Q0);
end
endmodule
```



```
initial begin
    iCLK=0;
    for(i=0;i<20;i++) begin
        #10; iCLK = ~iCLK;
    end
end
```

```
initial begin
    $dumpfile("ctr2bit_fsm.vcd");
    $dumpvars;
    rst=1;
    #10 rst=0;
    #125 rst = 1;
    #40 $finish;
end
```

Waveform 2 bit counter

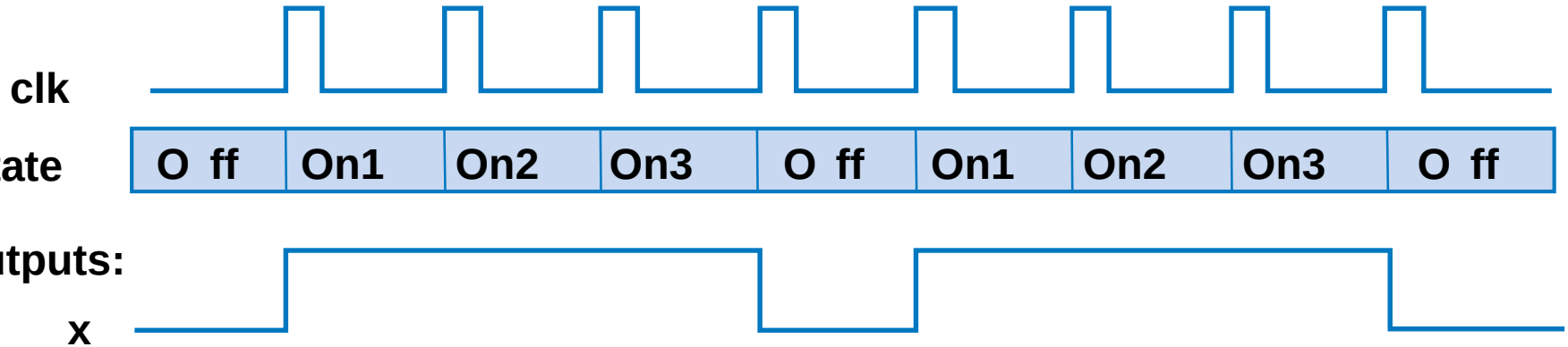


FSM Example 6

(General : sequence generator)

FSM Example: 0,1,1,1,repeat

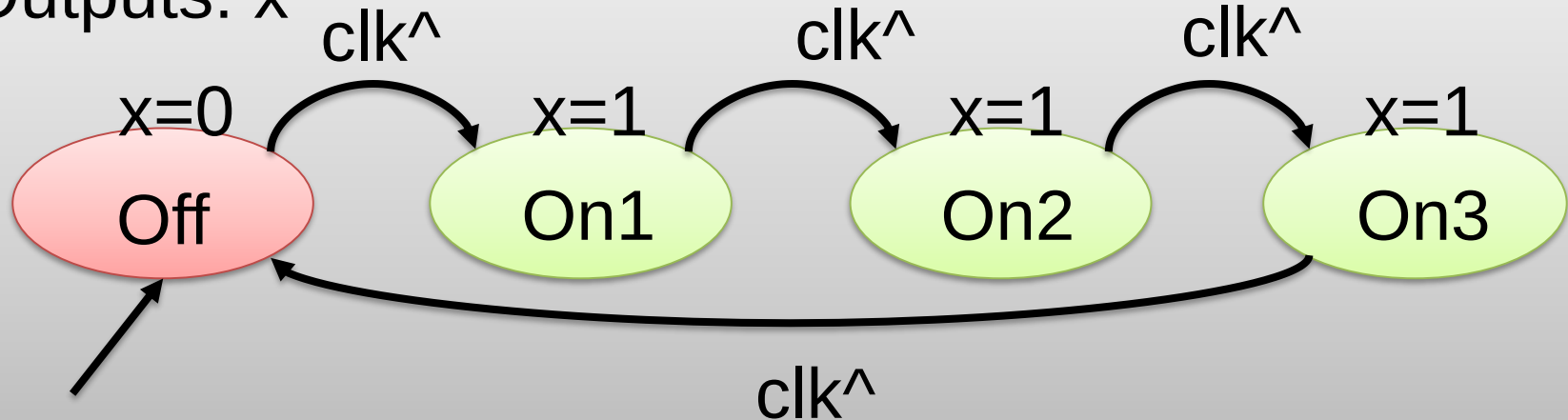
- Step1: Want 0, 1, 1, 1, 0, 1, 1, 1, ...
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states,
Transition on rising clock edge to next state



FSM Example: 0,1,1,1,repeat

- Step1: Want 0, 1, 1, 1, 0, 1, 1, 1, ...
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states, Transition on rising clock edge to next state

Outputs: x



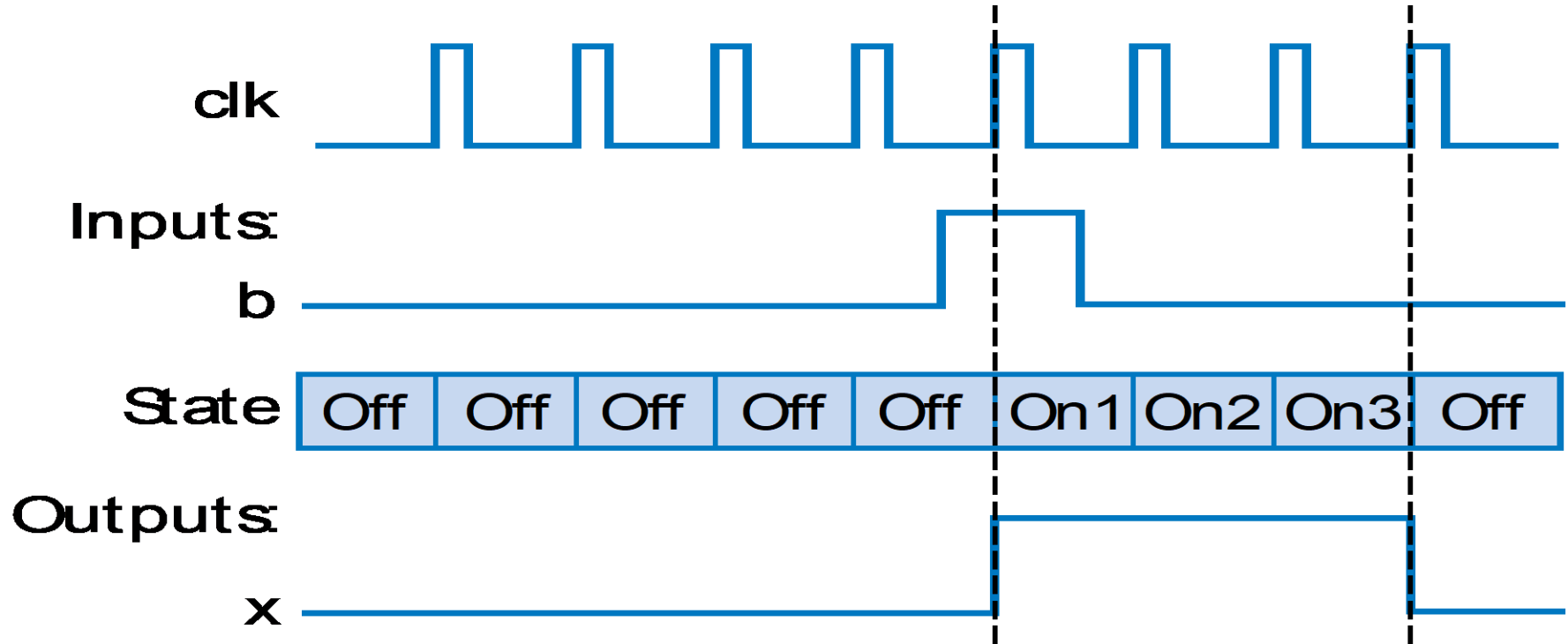
FSM Example 7

(General Example: Three-Cycles High Laser Timer)

Extend FSM to Three-Cycles High Laser Timer

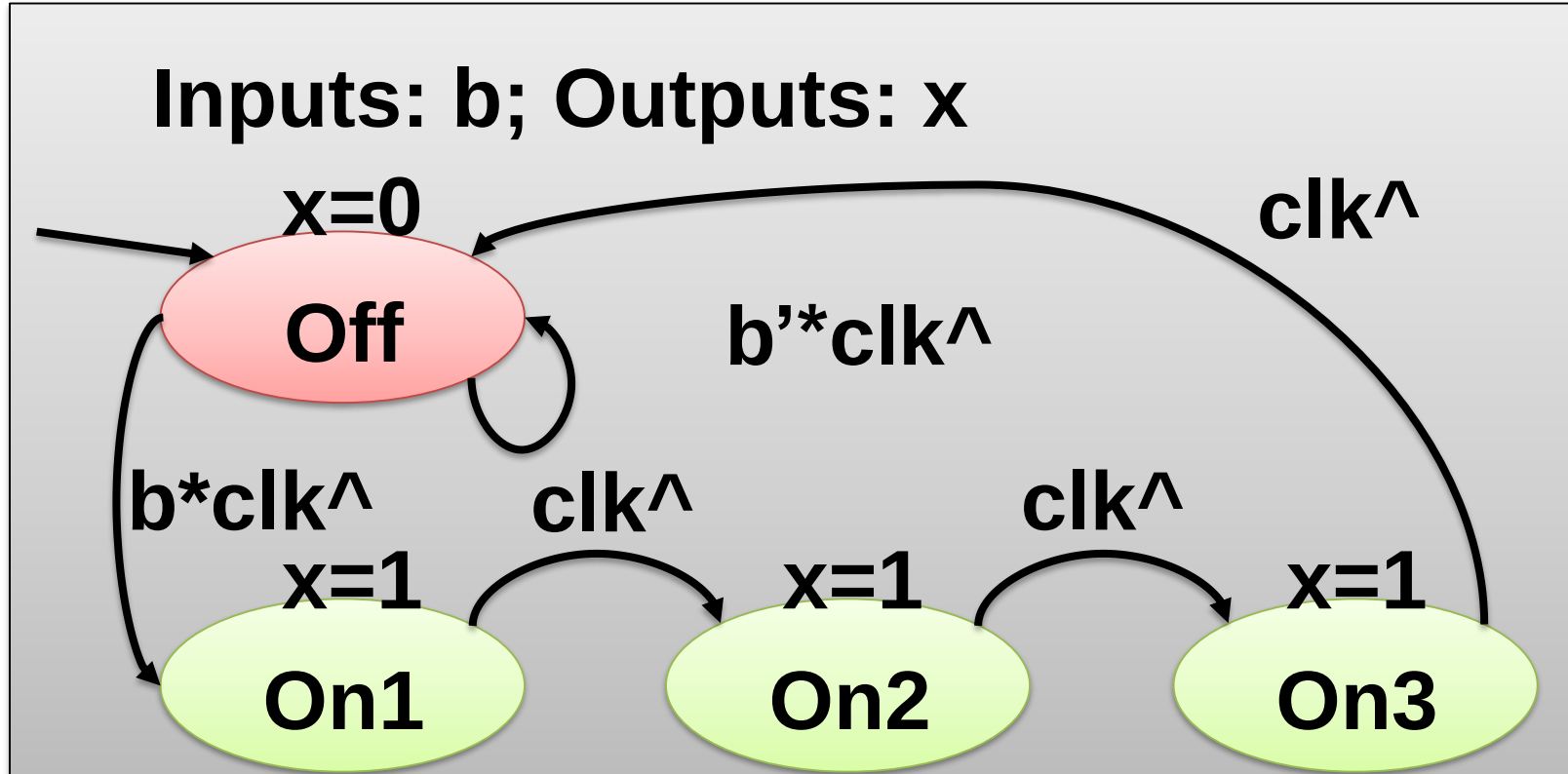
- Step1: Specify the problem in English like Stmts
 - Four states: Wait in “Off” state while b is 0 (b')
 - When $b=1$ (& rising clock edge), transition to On1
 - Sets $X=1$
 - On next two clock edges, transition to On2, then On3, which also set $x=1$
 - So $x=1$ for three cycles after button pressed

Extend FSM to Three-Cycles High Laser Timer



Specify: FSM of Three-Cycles High Laser Timer

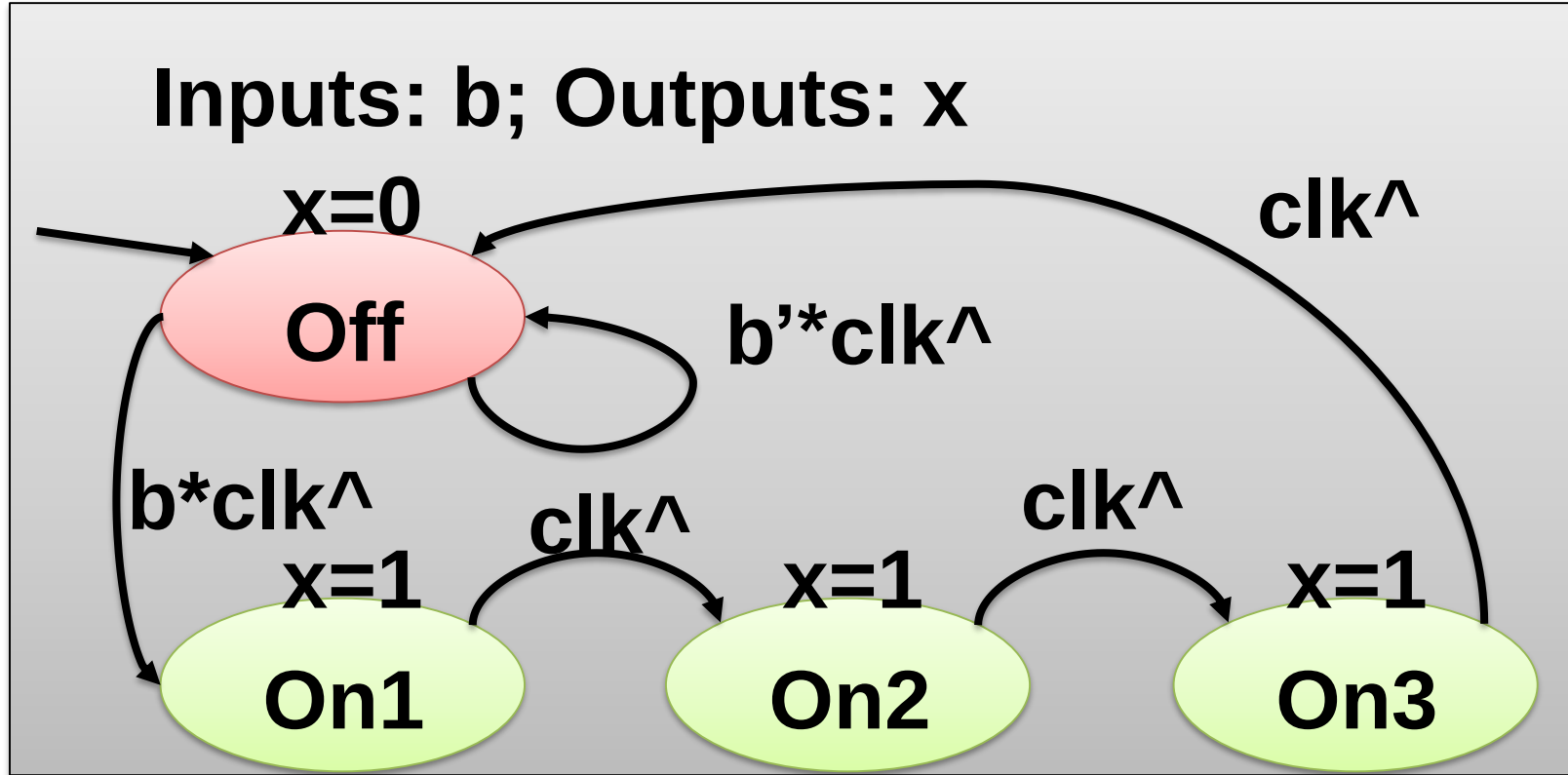
Step2: Model the problem using FSM



FSM Simplification: Rising Clock Edges Implicit

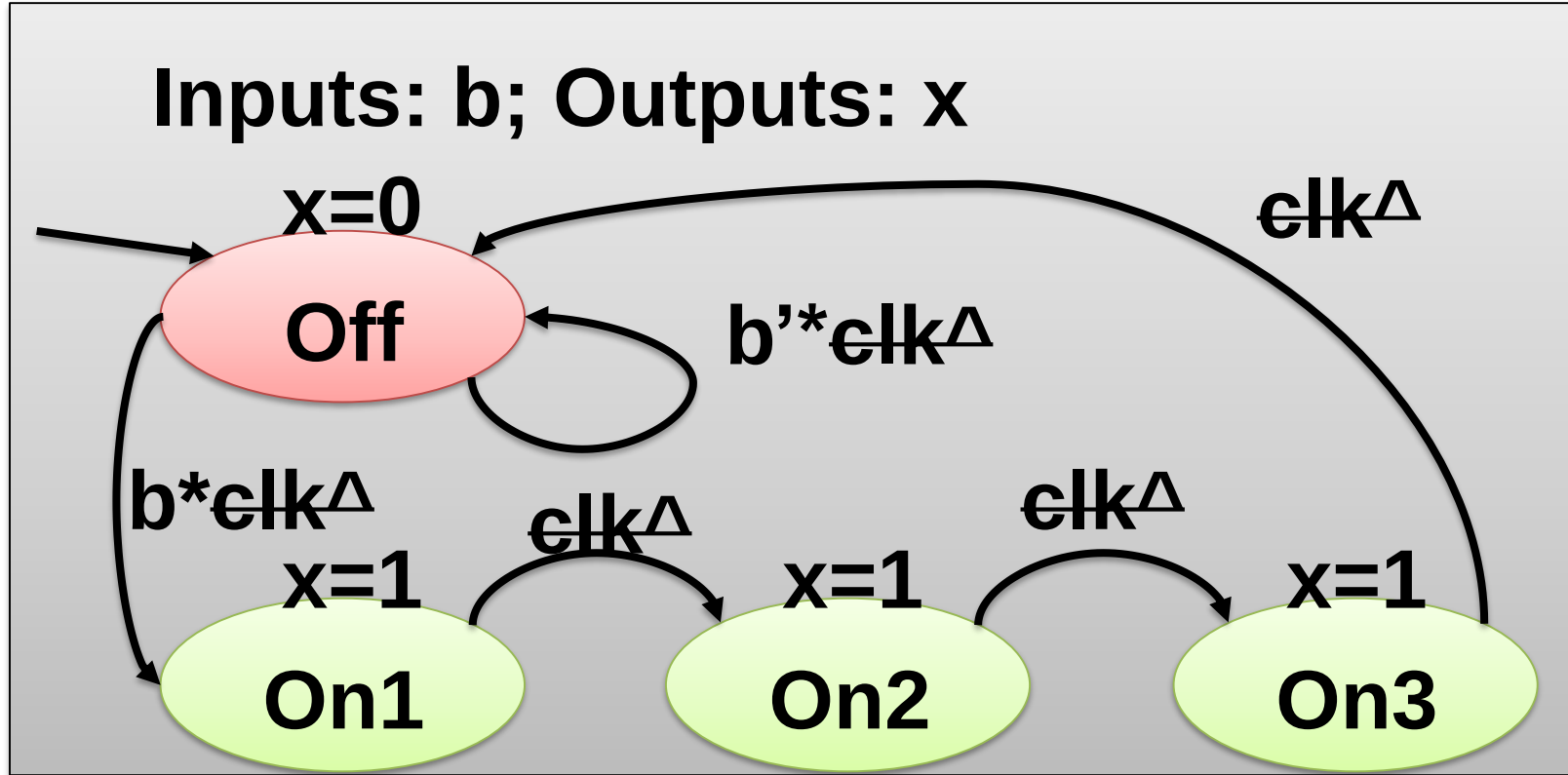
- Showing rising clock on every transition: **cluttered**
- Make implicit -- assume every edge has rising clock
- What if we wanted a transition *without* a rising edge
 - Asynchronous FSMs -- less common, and advanced topic
 - We consider *synchronous* FSMs
 - **All transition on rising edge**

Extend FSM to Three-Cycles High Laser Timer



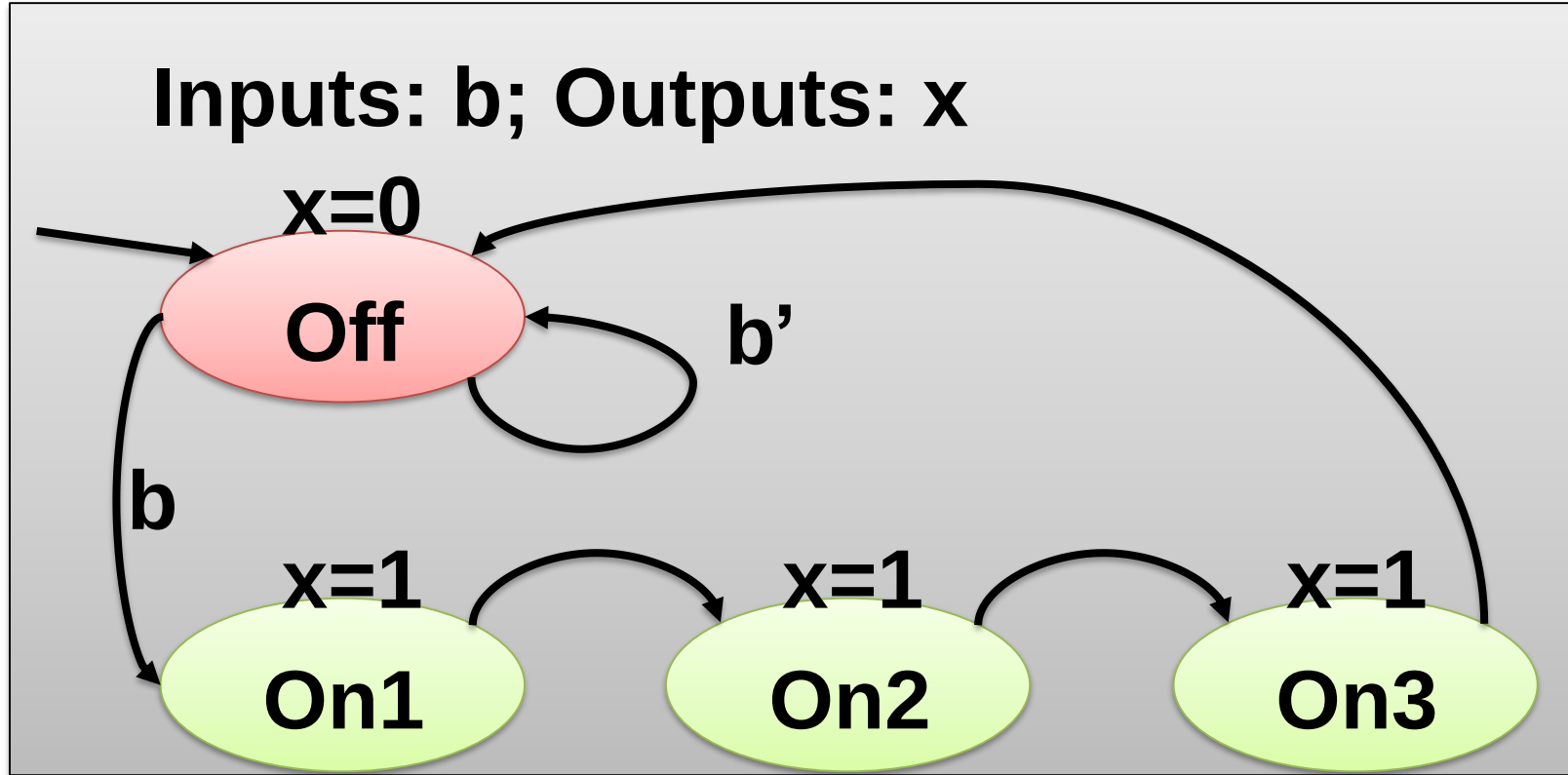
Note: Transition with no associated condition thus transitions to next state on next clock cycle

Extend FSM to Three-Cycles High Laser Timer



Note: Transition with no associated condition thus transitions to next state on next clock cycle

Extend FSM to Three-Cycles High Laser Timer



Note: Transition with no associated condition thus transitions to next state on next clock cycle

FSM Implementation

Step 4: Derive number of FF require

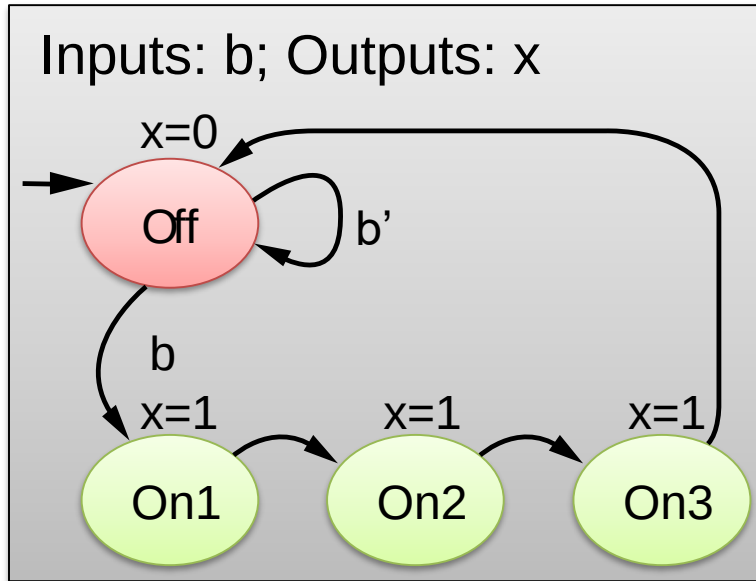
Four States: 2 FF

Step 5: State Encoding

Off: 00 On1: 01

On2: 10 on3: 11

Step 6: Derivation of Next State



PS	b	NS	X
00	0	00	0
00	1	01	0
01	x	10	1
10	x	11	1
11	x	00	1

FSM Implementation

PS	b	NS	X
00	0	00	0
00	1	01	0
01	x	10	1
10	x	11	1
11	x	00	1

Step 7: Derive for NS logic, OP logic

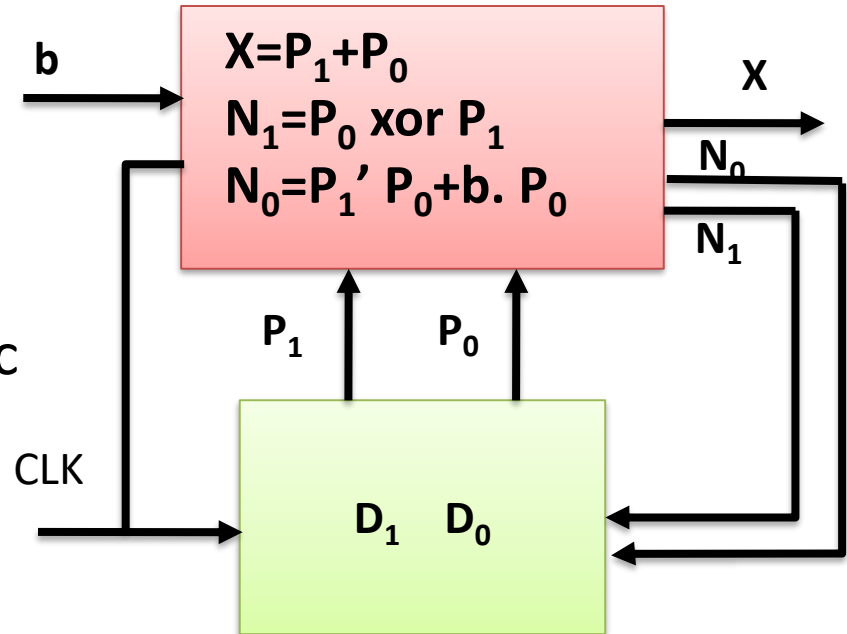
$$N_1 = P_0 \text{ xor } P_1$$

$$N_0 = P_1' P_0 + b \cdot P_0$$

$$X = P_1 + P_0$$

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



**Once we specify FSM for a
problem/system**

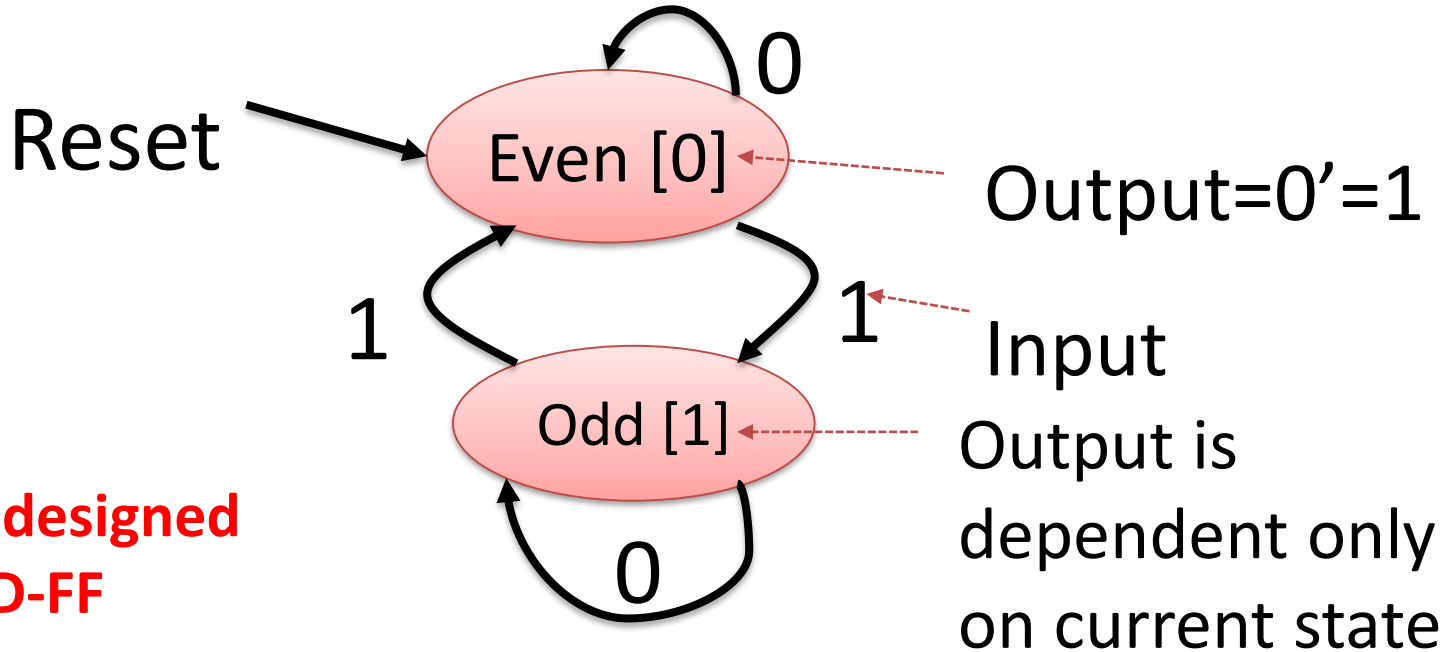
=== >

**Implementation is not
difficult**

FSM Example 8: Parity Encoder

FSM Example 8: Parity Encoder

- Input: 1 or 0 // entering as stream
- Output: output a 1 when total number of 1 is even

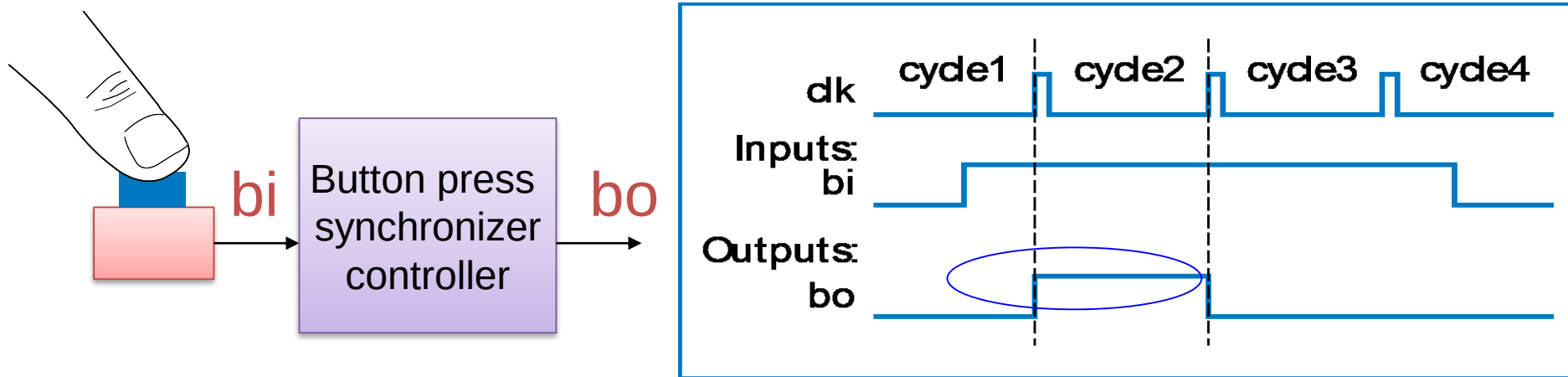


**T- FF : designed
using D-FF**

FSM Example 9: Button Press Synchronizer

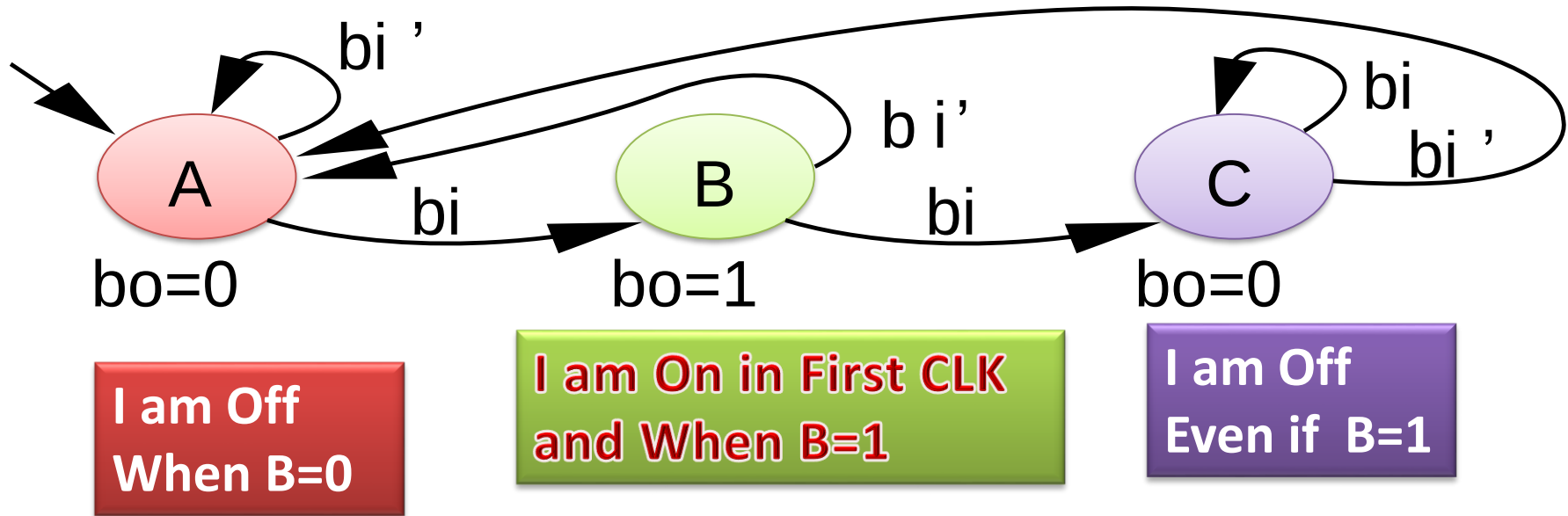
FSM : Button Press Synchronizer

- English Language Specification
- We want simple sequential circuit
 - Converts button press to single cycle duration
 - Regardless of length of time that button actually pressed



FSM : Button Press Synchronizer

FSM inputs: bi ; FSM outputs: bo



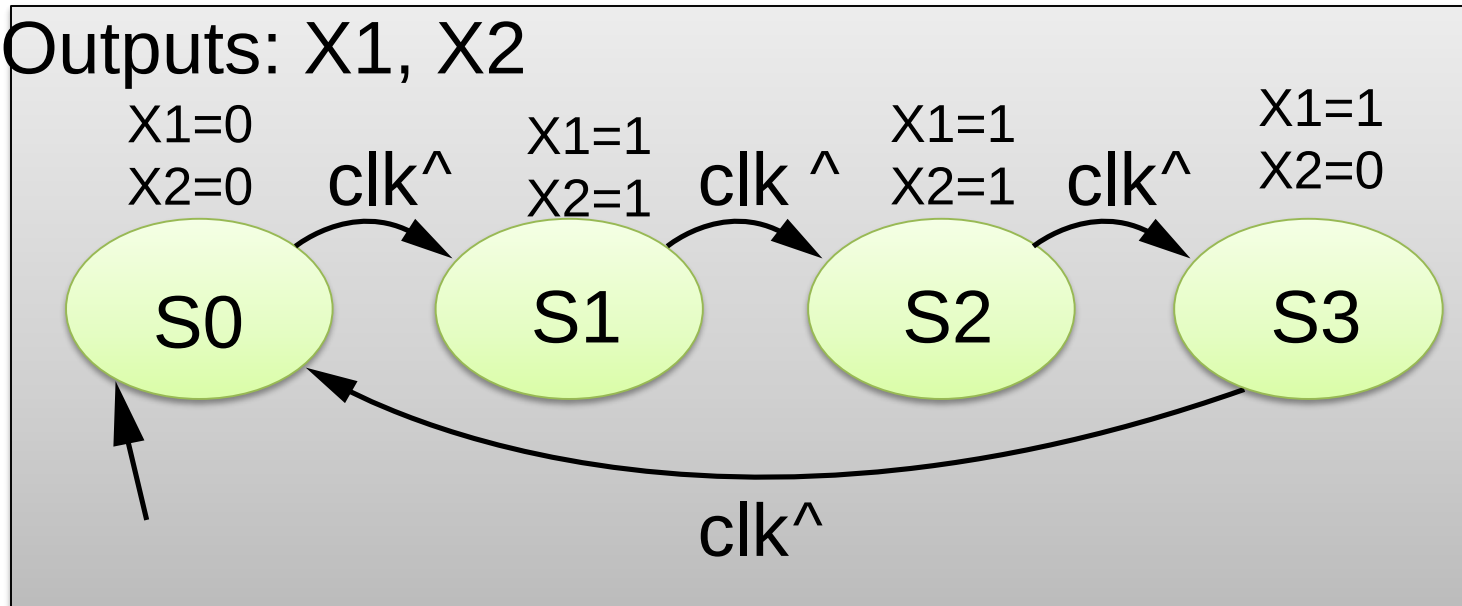
Step 1: Design FSM

FSM Example 10: Sequence generator

FSM Example 10: Sequence generator

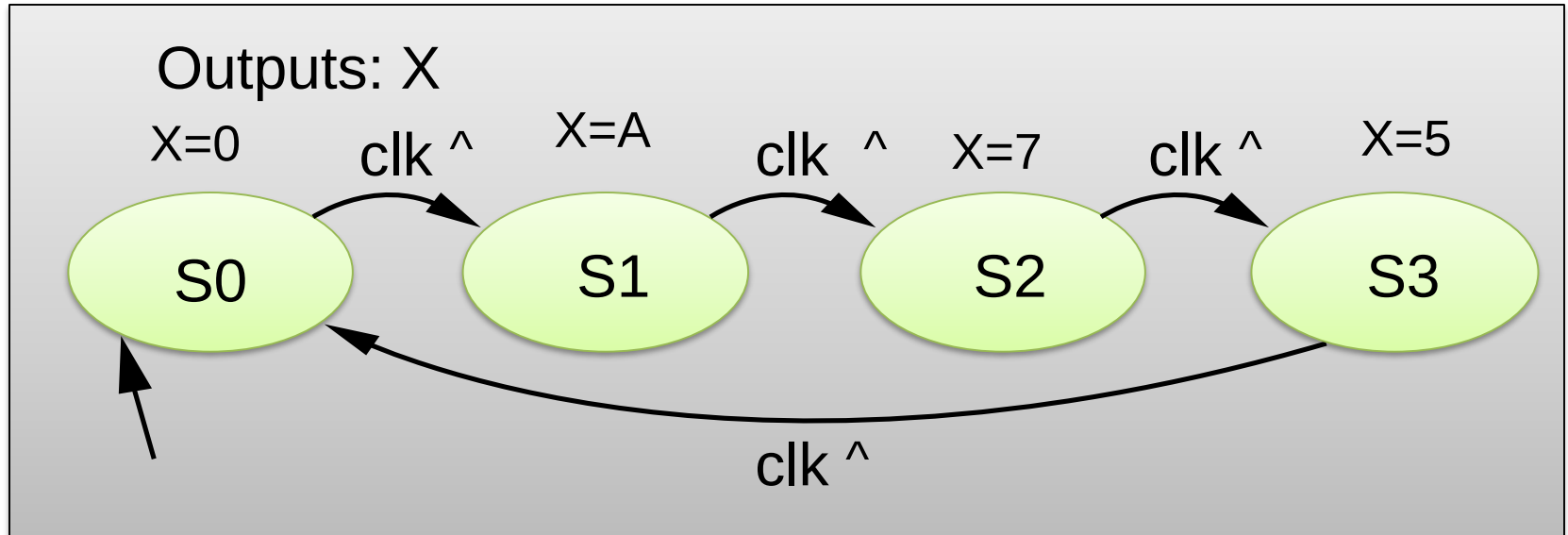
- Generate two output sequence
 - $X1 = 0111\dots$ repeat
 - $X2 = 0110\dots$ repeat

Outputs: $X1, X2$



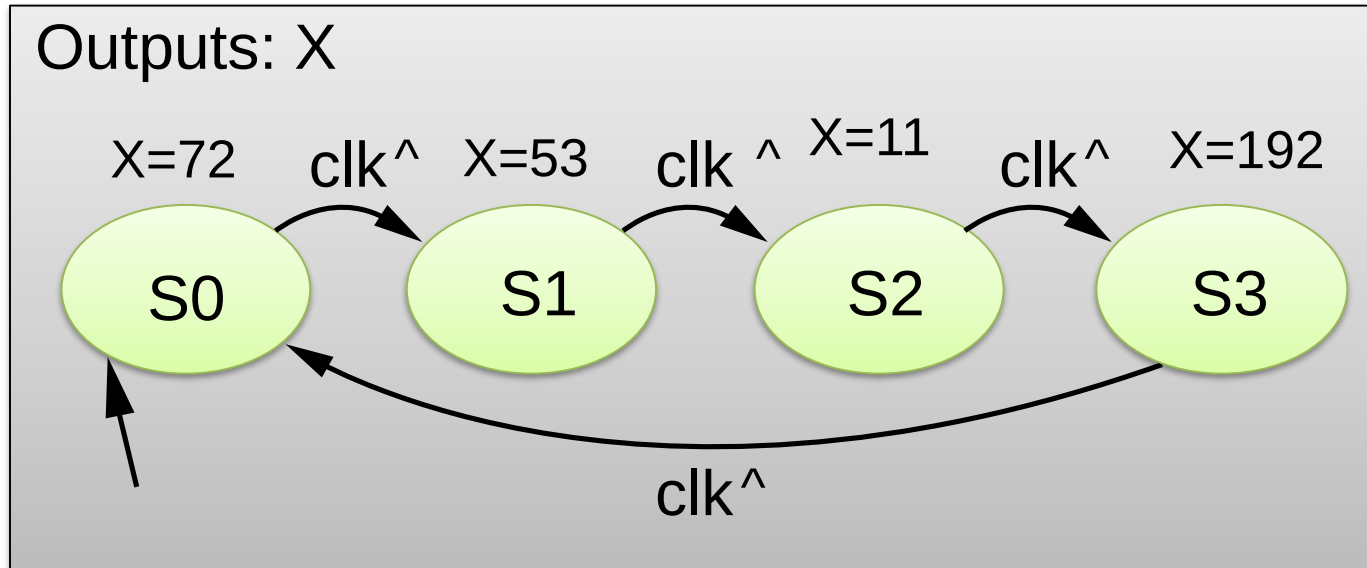
FSM Example 10-E1: Sequence generator

- Generate 4 bit integer output sequence
 - $X = 0, A, 7, 5 \dots$ repeat



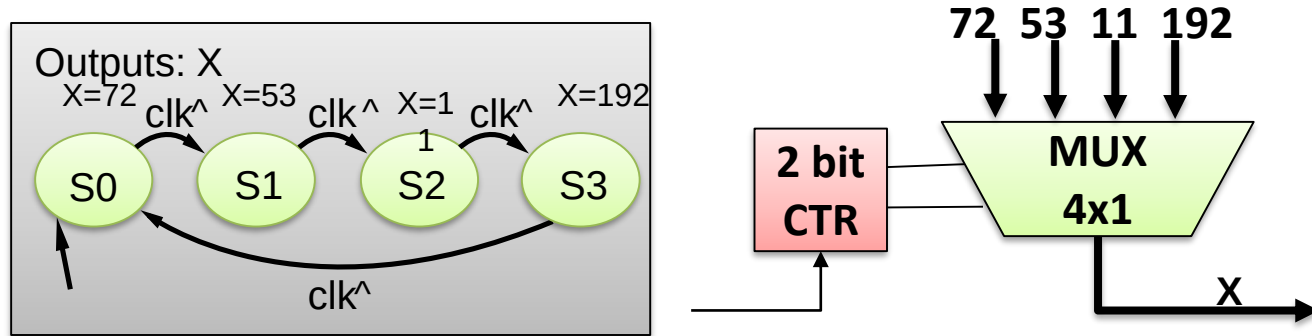
FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



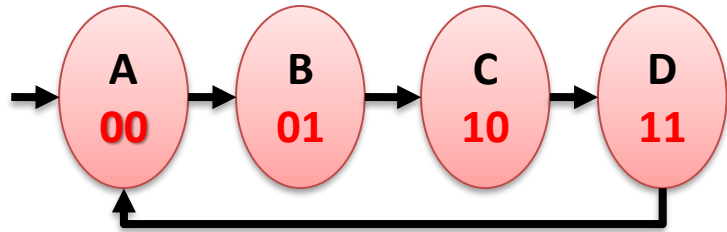
FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



A,B,C,D values can be hardwired

FSM Example 10-E2: Sequence generator



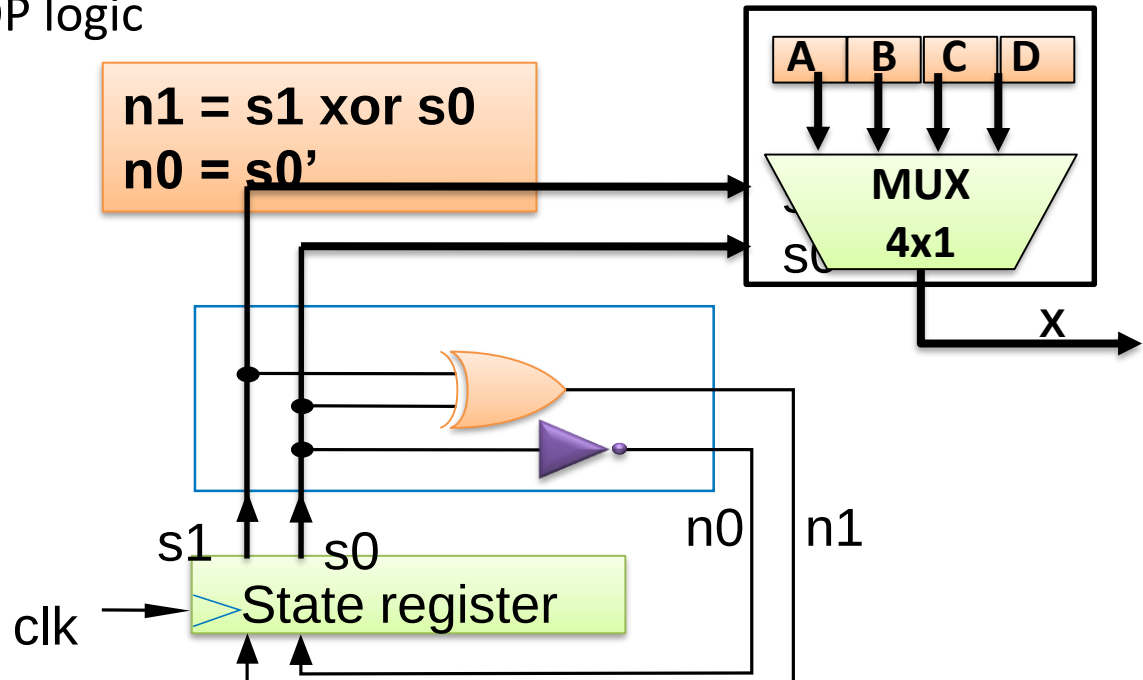
Step 7: Derive for NS logic, OP logic

NS : $n1 = s1 \text{ xor } s0$; $n0 = s0'$

	PS		NS	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
D	1	1	0	0

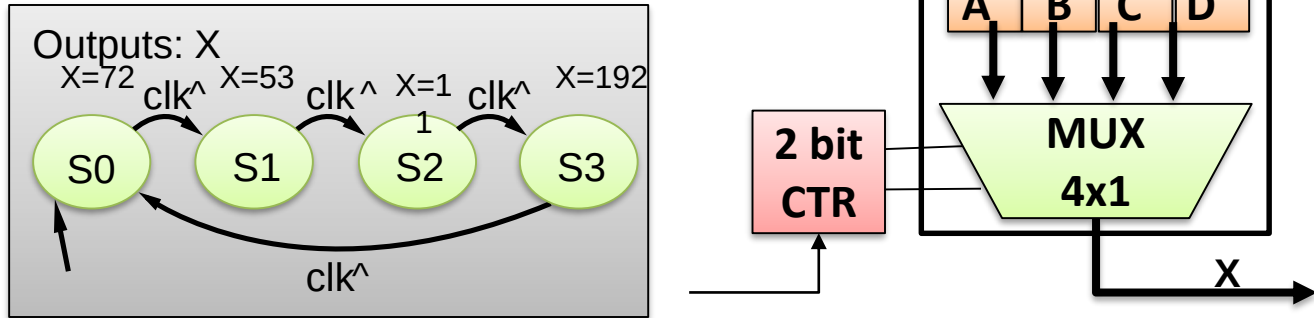
Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



A,B,C,D values can stored in 4 register
(With Mux it act as memory)

FSM Output logic
may be Simpler than
Register and Mux

Simpler to Implement
Output logic: Eight 2
input binary function

**Once we specify FSM for a
problem/system**

=== >

**Implementation is not
difficult**

Thanks